
Dual-Output AI Pair Programming for Trustworthy Coding: A User Study

Felix Wang¹ Ashu Adhikari¹

Abstract

The integration of Large Language Models (LLMs) into software development has accelerated engineering velocity, but it has also introduced a critical issue. Models frequently generate functionally correct but insecure code. Driven by automation bias, developers often integrate these outputs without sufficient scrutiny. To mitigate this, we propose “Dual-Output AI Pair Programming,” a novel interaction model that presents developers with two distinct code options to disrupt passive acceptance and trigger comparative evaluation. We evaluated this approach through a controlled pilot study ($N = 8$) within a simulated GitHub environment, utilizing tasks adapted from the BaxBench benchmark (Vero et al., 2025). While the pilot yielded no statistically significant improvement in vulnerability remediation, secondary metrics confirmed that the dual-output interface successfully induced “cognitive forcing,” evidenced by increased task duration and perceived difficulty. Qualitative analysis attributes the lack of improvement to the absence of distinct security cues across options. We therefore establish a validated protocol for a full-scale investigation that incorporates a *Differential Security* strategy to transform cognitive friction into effective security auditing.

1. Introduction

Integrating Large Language Models (LLMs) has completely transformed the software development process. State-of-the-art models (SOTA) now generate functional code proficiently, resolving over 70% of real-world GitHub issues (Jimenez et al., 2023). Driven by rapid delivery pressures

and automation bias, developers will increasingly adopt LLMs for code generation. For instance, GitHub’s 2025 Octoverse report notes that 80% of new developers use tools like Copilot within their first week. However, this reliance introduces critical vulnerabilities. While models often produce functional code, they frequently fail to meet security standards, with correctness and security coinciding only half the time (Vero et al., 2025). Furthermore, while developers naturally test and refine code until it functions properly, they need to deliberately seek out potential security flaws. This disparity creates a precarious operational environment.

Ideally, model advancements would eventually address both functional and security deficits, but such progress is slow. In the interim, the widespread commercial adoption of LLMs necessitates an immediate solution to the reliability gap. We therefore investigate whether Human-AI interaction design can compensate for model fallibility. Specifically, we posit that the interface itself can serve as a cognitive forcing function. In this paper, we introduce a “Parallel Pull Request” workflow that presents developers with two distinct code options rather than a single authoritative answer. By altering the presentation, we aim to disrupt the “mindless copy-paste” loop and trigger critical, comparative evaluation.

We position this work as the foundation of a broader research initiative. To accommodate the current scope while mapping our future trajectory, we explicitly distinguish between the *pilot study* completed for this course and the *long-term vision*. Consequently, we organize the subsequent sections of this paper as follows: first, we detail the execution and findings of the pilot, then discuss how these preliminary insights shape the design of the full-scale investigation.

Specifically, through the pilot study, we:

- **Develop an Analysis Framework:** Create a qualitative coding scheme for screen recordings that transforms raw video into structured data suitable for hypothesis testing.
- **Define Metrics:** Propose specific descriptive and inferential measures to quantify developer behaviour and security outcomes.
- **Provide Empirical Evidence:** Report preliminary findings regarding the efficacy of parallel suggestions.

¹Department of Computer Science, University of Waterloo, Waterloo, Canada. Correspondence to: Felix Wang <felix.wang@uwaterloo.ca>, Ashu Adhikari <ashu.adhikari@uwaterloo.ca>.

- **Refine the Protocol:** Identify experimental limitations and the formulation of mitigation strategies to solidify the rigour of the full-scale investigation.

Building on this foundation, the full-scale study aims to contribute the following:

- **Paradigm Shift:** A movement beyond merely auditing LLM code security toward empowering humans to remediate it.
- **Novel Interaction Model:** The design and empirical evaluation of a system specifically engineered to manage imperfect AI outputs safely.
- **Design Guidelines:** The derivation of actionable principles to inform the architecture of future coding assistants.
- **Quantification of Benefit:** The establishment of a lower bound on the utility of providing multiple code suggestions.

2. Related Works

2.1. Benchmarking LLM Code Generation

Existing literature has extensively evaluated the functional capabilities of LLM-generated code. Current benchmarking indicates that SOTA models are proficient at producing functional code, though success rates vary by complexity. Strategies such as repeated sampling can boost performance on standard functional tasks from roughly 28% to over 70% (Chen et al., 2021), while fine-tuned models achieve accuracies exceeding 80% on complex problem sets (Austin et al., 2021). Moreover, recent evaluations demonstrate high ecological validity by testing against real-world GitHub issues, where models have successfully resolved up to 70% of the problems (Jimenez et al., 2023).

However, functional success does not imply security. While many benchmarks assess correctness, few rigorously evaluate the security posture of the generated artifacts. To address this gap, Vero et al. recently introduced BaxBench, a benchmark specifically designed to audit the security of backend application generation. Using expert-written exploits across 28 scenarios and 14 frameworks, they identified a critical “reliability gap.” They found that, on average, exploits compromised approximately half of the functionally correct programs generated by each model. Even the highest-performing flagship models produced secure and correct code only 37% of the time. This result underscores that LLM-generated codes require significant human oversight to prevent the deployment of vulnerable software, which directly motivates our study.

2.2. Human-AI interaction for Code Security

While the technical capabilities of LLMs are well-documented, a growing body of research has examined the human factors of neural code generation. For instance, Ziegler et al. and Vaithilingam et al. examine productivity and workflow integration, while Barke et al. and Fakhoury et al. investigate interaction dynamics and intent alignment (Barke et al., 2022; Fakhoury et al., 2024; Vaithilingam et al., 2022; Ziegler et al., 2022). Despite these advances, most of the Human-AI interaction has largely prioritized usability over security. We identified only three user studies that specifically evaluate the impact of LLM-generated code on security behavior, which we detail below.

Perry et al. conducted the first comprehensive user study ($N = 47$) examining security across Python, JavaScript, and C (Perry et al., 2023). They found that participants with AI access wrote significantly less secure code than those without, particularly in tasks involving cryptography and SQL injection. Crucially, they observed an inverse relationship between security and trust: participants with AI access were more likely to believe that their insecure code was secure.

In contrast, Sandoval et al. ($N = 58$) and Asare et al. ($N = 25$) reported neutral to positive effects in lower-level contexts (Sandoval et al., 2023; Asare et al., 2024). Sandoval et al. tasked students with implementing a C-based linked list and found that AI-assisted users produced critical security bugs at a rate no greater than 10% higher than the control group, concluding that LLMs did not introduce significant new security risks in that specific domain. Asare et al. extended this by observing that while AI access had no effect on simple problems, it actually correlated with more secure solutions for complex programming tasks, potentially because the model maintained security priorities when human cognitive load increased.

Collectively, these studies illuminate the complex, context-dependent nature of AI-assisted security. They establish a crucial baseline: developers are susceptible to over-reliance, yet capable of security vigilance under specific conditions. We build on this diagnostic foundation by shifting the focus from assessment to intervention. Rather than simply observing how current tools impact security, we investigate how a novel interaction design can actively improve it. By proposing and evaluating the “Parallel Pull Request” model, we aim to determine if the interface itself can disrupt the patterns of over-trust identified in prior work and serve as a cognitive forcing function for security.

2.3. Cognitive Models of Program Comprehension

To rigorously interpret developer behaviour, we ground our methodology in foundational cognitive models of program

comprehension. Littman et al. identified two distinct macro-strategies: a “systematic strategy,” where a programmer traces data flow to build a comprehensive causal model of the system, and an “as-needed strategy,” where understanding is localized and limited to static knowledge (Littman et al., 1987). At the micro-level, Letovsky characterizes comprehension as an opportunistic cycle of “inquiry,” where developers formulate questions and conjectures to connect code to their existing knowledge base (Letovsky, 1987). We posit that current single-stream LLM interfaces actively inhibit these inquiry cycles by providing immediate, unverified closure rather than provoking analysis. Consequently, these frameworks provide the theoretical basis for our investigation. They allow us to measure whether our intervention shifts developers from passive integration toward active scrutiny, thereby revealing opportunities for targeted design interventions.

3. Methods

3.1. Participants: Recruitment and Screening

Participants for this study were recruited online via the university mailing lists. Participants who expressed interest in our study were asked to fill consent form and a screening form to determine their eligibility for the study. Our selection criteria were based on age (between 18 - 64 years old), employment history (no affiliation with the development of ChatGPT or OpenAI), and particularly emphasized an intermediate to advanced level of Python coding experience (at least one year of coding experience in Python) and high Code Generation Tools (CGTs) usage frequency. The study was approved by the Research Ethics Board (REB) of the University of Waterloo.

Like prior studies in this area, participants were not screened based on system security background because the goal of this study was to investigate the impact of CGT output quantity on code security among ordinary developers, regardless of system security background. The effect of participant security experience is considered representative of the heterogeneity found in real-world software teams. However, we do consider the idea of exploring the effects of participant security experience on the code security outcome across the single-output and multi-output conditions as an interesting future study.

A total of 9 individuals expressed interest in taking part in the study. One candidate did not pass our screening process, resulting in 8 out of 9 people completing all stages of the study for a completion rate of 88.9%. All of the user study sessions were conducted using Zoom and Microsoft Teams, lasting 60 - 220 minutes ($M = 100.17$). The majority of our participants reported to have 5+ years of coding experience ($n = 6$) with 3+ years specializing in Python

Table 1. Participant Summary

Dimension	Attribute	<i>n</i>
Coding Experience	1 - 3 years of Coding Experience	2
	5 + years of Coding Experience	6
Python Coding Experience	1 - 3 years of Python Coding Experience	2
	3 - 5 years of Python Coding Experience	3
	5 + years of Python Coding Experience	3
FastAPI Coding Experience	I have heard of it before	4
	I have used it once or twice before	2
	I have used it for more than 3 projects before	2
CGTs ¹ Usage	I have used them regularly (daily or weekly)	8
Education Background	Master’s degree (or in progress)	2
	PhD (or in progress)	6
Security Background	Rare knowledge	2
	Self-learned or informal training	2
	Professional experience	1
	Formal education	1
	Formal education and professional experience	1
	Other	1

¹ CGTs = Code Generation Tools.

($n = 6$), have at least heard of FastAPI before ($n = 8$), use CGTs regularly ($n = 8$), have held or were pursuing a PhD ($n = 6$), possessing certain knowledge in system security ($n = 6$).

3.2. Procedure

To empirically evaluate whether presenting alternatives shifts developer behaviour from passive acceptance to critical selection, we designed a controlled experiment within a simulated GitHub workflow. In this environment, participants assumed the role of Repository Maintainers tasked with reviewing Pull Requests (PRs) submitted by an AI-assisted contributor.

We used pre-generated codes rather than employing live, stochastic LLM generation. This decision ensures experimental control. By holding the type, location, and visibility of security vulnerabilities constant, we eliminate the confounding variables introduced by the variable nature of model outputs. This consistency facilitates rigorous quantitative analysis while preserving the ecological validity of a standard asynchronous review process.

We employed a within-subjects design, counterbalanced via a Latin Square to mitigate learning and order effects. The experiment contrasted two distinct interface conditions:

- **Condition A (Single-Output Control):** Participants reviewed a single Pull Request addressing a specific issue, simulating the standard linear review workflow.

- **Condition B (Dual-Output Control):** Participants reviewed two parallel Pull Requests offering alternative solutions to the same issue, forcing a comparative review process as illustrated in Figure 1.

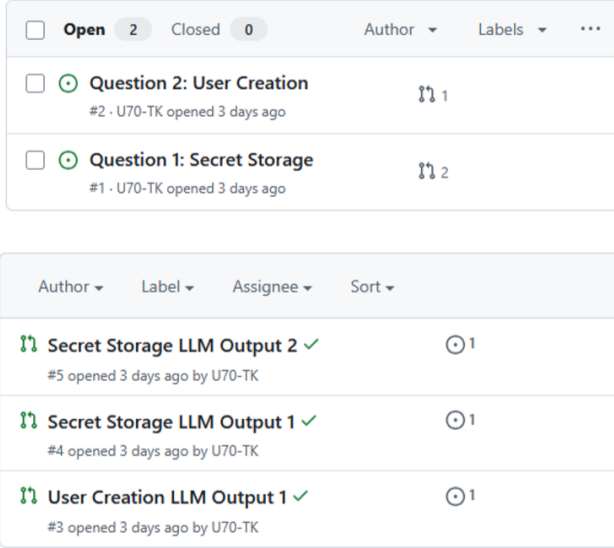


Figure 1. The Simulated GitHub Interface (**Question 1: Secret Storage** task illustrates the Dual-Output condition, **Question 2: User Creation** illustrates the Single-Output control.)

Consequently, this design isolates the effect of the interaction model, allowing us to determine if the the presentation of multiple options heightens security consciousness or merely imposes unproductive cognitive load.

3.3. Programming Problems

To ensure task validity, we adapted scenarios from the BaxBench benchmark, recently introduced by Vero et al (Vero et al., 2025). BaxBench evaluates backend code generation by testing solutions from 10 state-of-the-art LLMs across 28 scenarios against expert-written security exploits. From this dataset, we selected two scenarios representing distinct but critical backend functions: User Creation (Identity Management) and Secret Storage (Sensitive Data Handling).

We prioritized these tasks because they expose the application to identical, high-impact vulnerabilities recognized by industry standards. Specifically, both scenarios target Improper Access Control (CWE-284) and Insufficiently Protected Credentials (CWE-522), which rank #1 and #6 in the OWASP Top 10 (OWASP Foundation, 2025a;b). They also involve SQL Injection (CWE-89), currently the #3 vulnerability in the MITRE Top 25 (The MITRE Corporation, 2024). This shared threat surface creates six distinct vulnerability vectors (2 tasks $\times$ 3 CWEs). Empirical data from the benchmark reveals that across these vectors, model vulnerability rates fluctuate between 0% and 100% (Vero et al., 2025). This extreme variability confirms that detec-

tion is neither universally trivial nor impossible, ensuring a balanced difficulty curve for human scrutiny. Finally, we limited the scope to two scenarios so that participants should be able to complete the review within a 60-minute timeframe, keeping the total experimental session under 90 minutes.

4. Experimental Evaluation

With the reliability gap identified in the literature and our experimental setup defined, we formulated the following research questions to guide our evaluation.

RQ1: Does the dual-output strategy help developers write more trustworthy code with fewer vulnerabilities?

The primary objective of this study was to determine if the “Parallel Pull Request” interface reduces the acceptance of insecure code. We defined *Security Performance* as an ordinal score ranging from 0 to 3 for each task. This score represents the count of specific vulnerabilities (SQL Injection, Improper Access Control, and Insufficiently Protected Credentials) successfully identified and remediated by the participants.

To evaluate overall efficacy, we compared the distribution of *Security Performance* scores (0–3) between the Single-Output (Control) and Dual-Output (Experimental) conditions using the Mann-Whitney U test. This non-parametric test allowed us to determine if the dual-output interface significantly shifted the distribution toward higher security compliance.

To understand if the interface aided in detecting specific types of flaws, we compared the remediation rates of each CWE using Fischer’s exact test. This revealed if the intervention was particularly effective for specific classes of vulnerabilities, such as injection risks, while potentially having a different impact on logic errors.

For this pilot study, we utilized this scoring system to validate the sensitivity of our metrics and establish baseline performance. In the full-scale investigation, we will rigorously evaluate the intervention using these statistical approaches (or refined alternatives) to definitively assess the intervention’s success.

RQ2: How does the dual-output workflow impact developer review behaviour and decision-making strategies?

The secondary objective was to understand the cost and mechanism of the intervention. While we hypothesized that parallel options trigger necessary scrutiny, we want to determine if this improved security comes at the expense of prohibitive inefficiency or excessive cognitive load.

Efficiency and Utility: To address the aspect of efficiency, we compared the *Time to Completion* across conditions.

We anticipated that reviewing two options would inherently increase task duration; the critical question was whether this increase remained within acceptable bounds for a standard workflow. To gather insights into utility, we analyzed post-task questionnaire ratings regarding helpfulness, decision confidence, and output preference. For this pilot, these measures served as a sanity check to validate our instrumentation; the full-scale investigation will use them to formulate definitive design guidelines.

Behavioural Analysis Framework: To elucidate the cognitive mechanisms underlying these outcomes, we recorded participant screens during the task. We sought to map the decision-making workflow: specifically, whether participants employed serial or parallel comparison strategies, the timing of vulnerability detection (e.g., during initial scan vs. detailed comparison), and potential indicators of comparison fatigue.

Macro-Level Phases: Drawing on the cognitive frameworks of Littman et al., we segmented the session into five Macro-Level Phases to map the “Shape of the Session.” This segmentation allowed us to compare how the Dual-Output interface altered the fundamental workflow:

- **Orientation Phase:** Begins when the task description is opened and ends when the first PR is clicked. This establishes the baseline comprehension time for the issue itself.
- **Exploration Phase:** The initial scan of the provided code. We interpret high scroll velocity as structural skimming and low velocity as logical reading.
- **Comparison Phase (Dual-Output Only):** The period where both PRs have been opened at least once, and the user alternates between them. This is the critical window for analyzing divergence in attention.
- **Review Phase:** The period of deep engagement, characterized by highlighting, commenting, and code editing.
- **Resolution Phase:** The final actions taken to merge, close, or provide summary feedback on the PRs.

Micro-Interaction Codebook: Complementing this structural analysis, we applied a micro-interaction codebook inspired by Letovsky’s model of inquiry. We coded specific screen artifacts as proxies for cognitive states within each macro-phase:

1. Orientation Phase

- **ORI_ISSUE:** Click to open and read the issue description.
- **ORI_FILE_DIFF:** Click to view the file diff in the browser.
- **ORI_CHECKOUT:** Clone the repository locally and checkout the PR branch to review the code.

2. Exploration Phase

- **EXP_FAST:** Rapid scrolling ($> N$ lines/sec). Indicates structural skimming.
- **EXP_SLOW:** Granular scrolling ($< N$ lines/sec). Indicates logical processing.
- **EXP_JUMP:** Use of “Go to Definition” or “Find” to navigate specific symbols.

3. Comparison Phase

- **FOCUS_PR1/PR2:** The viewport displays the first or second PR, respectively.
- **SWITCH_1TO2/2TO1:** Toggling active focus between PRs.
- **COMP_SIDE_BY_SIDE:** Arrangement of windows to view both PRs simultaneously.
- **COMP_REVERIFY:** A rapid toggle sequence (e.g., $PR1 \rightarrow PR2 \rightarrow PR1$ in $< t$ seconds), indicating a discrepancy check.

4. Review Phase

- **REV_HIGHLIGHT:** User highlights text with the cursor (proxy for reading focus).
- **REV_EXTERNAL:** User switches to an external browser tab (e.g., documentation).
- **REV_COMMENT:** User drafts a comment for specific code feedback.
- **REV_CODE:** User edits the code directly.
- **REV_TEST:** User runs the provided test suite.

5. Resolution Phase

- **RES_COMMENT:** Typing a summary comment on the PR.
- **RES_MERGE_CLOSE:** Executing the merge and closing the issue.

For the pilot study, our primary objective was methodological validation. We sought to determine whether the proposed micro-interaction codebook accurately captures developer intent or requires modification.

In the full-scale investigation, we will utilize this validated framework to conduct a mixed-methods analysis. We will use observational notes for thematic analysis of user strategies and timestamped logs for quantitative metrics. Specifically, for the Dual-Output condition, we will quantify cognitive allocation using the Attention Symmetry Index (ASI):

$$ASI = 1 - \frac{|T_{PR_1} - T_{PR_2}|}{T_{PR_1} + T_{PR_2}} \quad (1)$$

Where T_{PR_x} denotes the total time engaged with Pull Request x . An ASI value of 1.0 signifies perfectly balanced attention, indicating active comparison. Conversely, a value

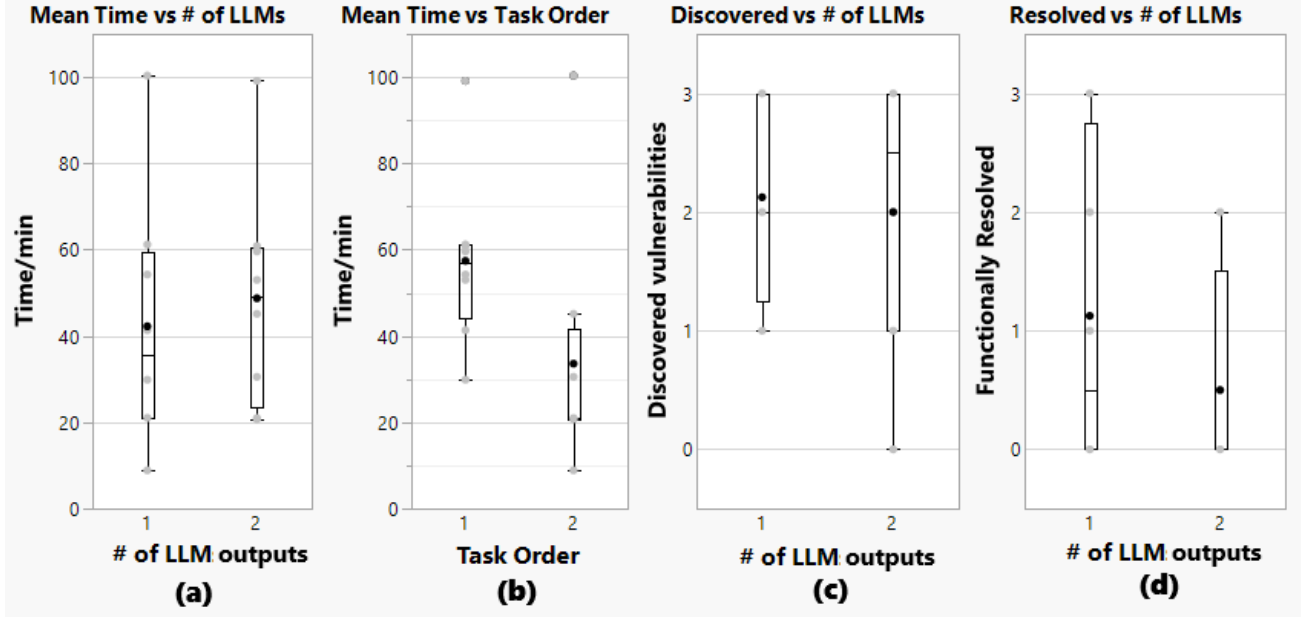


Figure 2. Box and Whisker plot of (a) Time of completion (mins) vs number of LLM outputs (b) Time of Completion (mins) vs Task Order (c) number of Discovered vulnerabilities vs number of LLM outputs and (d) number of Functionally Resolved vulnerabilities vs number of LLM outputs. The dark circle represents the mean, and the gray circle represents each participant’s performance.

near 0.0 indicates fixation on a single option, suggesting that the participant effectively treated the dual-output interface as a standard single-stream workflow.

5. Results

We present the findings from our pilot study ($N = 8$), organized by quantitative outcomes regarding security and efficiency, followed by a qualitative analysis of developer workflow.

5.1. Quantitative Findings: Security and Efficiency

Before detailing the comparative results, we must acknowledge a critical, unanticipated discrepancy between the estimated and actual task duration. While the experimental design targeted a 60-minute total session, participants averaged over 90 minutes ($M = 91$ min, $SD = 46$) to complete both code reviews as seen in Figure 2(b). Analysis reveals that this duration was heavily influenced by the order of tasks rather than the experimental condition. Participants spent significantly less time on their second task ($M = 34$ min, $SD = 30$) compared to their first ($M = 57$ min, $SD = 20$). Chi-Square approximation (Kruskal-Wallis) confirms this difference is statistically significant ($\chi^2 = 4.864$, $df = 1$, $p = 0.027$). Our observation and this sharp decline suggests the onset of fatigue, leading to rushed reviews in the second half. Had participants maintained their initial level of scrutiny for the second task, total session times would have been prohibitive.

Originally, we sought to measure Security Performance as a singular metric comprising identification and remediation. However, during the study, participants frequently expressed low confidence in their ability to implement fixes within the FastAPI framework, often limiting their feedback to high-level comments rather than code changes. To decouple code fluency from security awareness, we bifurcated the security metric into two distinct variables: Discovered Vulnerabilities (the count of issues correctly identified in comments) and Functionally Resolved (the count of issues successfully fixed in the code).

5.1.1. IMPACT ON SECURITY OUTCOMES (RQ1)

Statistical analysis reveals that the dual-output interface did not yield a statistically significant improvement in security outcomes during this pilot.

Discovered Vulnerabilities: As shown in Figure 2(c), the mean number of vulnerabilities discovered was nearly identical between the Single-Output ($M = 2.13$) and Dual-Output ($M = 2.00$) conditions. A Wilcoxon/Kruskal-Wallis test confirmed no significant difference ($Z = 0.00$, $p = 1.00$).

Functionally Resolved: Similarly, the remediation rate did not improve with the parallel interface (Figure 2)(d). In fact, the control group achieved a slightly higher mean resolution score ($M = 1.13$) compared to the experimental group ($M = 0.50$), though this difference was not statistically significant ($Z = -1.03$, $p = 0.30$).

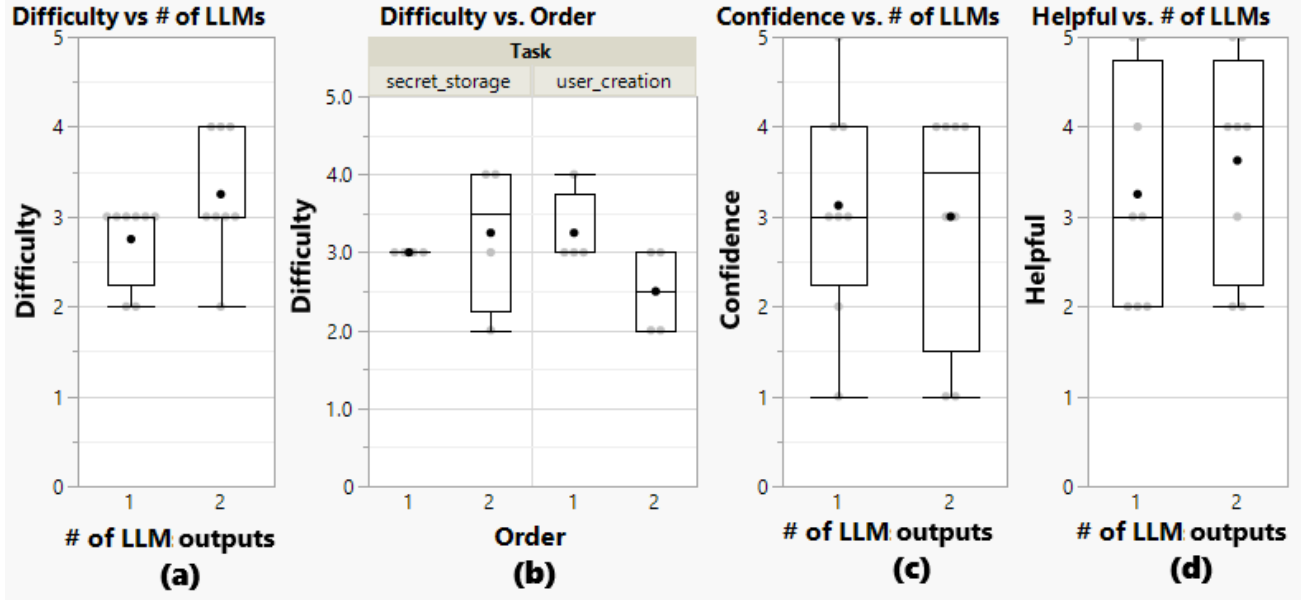


Figure 3. Box and Whisker plot of (a) Perceived difficulty of the task vs number of LLM outputs (b) Perceived difficulty of the task vs Task Order (c) Perceived confidence in the solution vs number of LLM outputs and (d) Perceived helpfulness vs number of LLM outputs. The dark circle represents the mean, and the gray circle represents each participant’s performance.

Vulnerability Specificity: To determine if specific types of flaws were easier to detect in a dual-output setting, we examined performance against individual Common Weakness Enumerations (CWEs). The number of LLMs provided had no impact on the discovery of specific vulnerabilities. Participants detected SQL Injection (7 vs. 6), Unprotected Credentials (7 vs. 6), and Improper Access Control (3 vs. 4) at statistically similar rates across conditions (Fisher’s exact tests, all $p = 1.00$).

5.1.2. IMPACT ON EFFICIENCY (RQ2)

Regarding workflow efficiency, we analyzed the *Time to Completion* to assess the cost of reviewing parallel options. As depicted in Figure 2(a), the Dual-Output condition resulted in a slightly higher mean completion time ($M = 49$ min) compared to the Single-Output control ($M = 42$ min). However, this increase of approximately 6.5 minutes was not statistically significant ($Z = 0.26, p = 0.79$). This suggests that while the dual-output workflow imposes a slight temporal overhead, it does not drastically prohibit rapid review cycles.

These results indicate that simply presenting two options does not automatically trigger superior security performance among novice users (students) with limited framework familiarity. The lack of statistical significance across all measures suggests that the interface alone may not be a sufficient forcing function without additional scaffolding, or that the sample size ($N = 8$) was insufficient to detect smaller effects.

5.2. Post-Task Questionnaire: User Perception

To complement the objective performance metrics, we analyzed participant responses from the post-task questionnaire regarding their subjective experience. Participants rated specific aspects of the interaction on a 5-point Likert scale (1= Low, 5= High). Figure 3 shows the results.

Perceived Utility and Helpfulness: Despite the lack of statistical significance in security outcomes, participants perceived the Dual-Output interface as marginally more helpful ($M = 3.63$) than the Single-Output control ($M = 3.25$). This suggests that while participants struggled to effectively utilize the alternative options to fix code, they appreciated the availability of a second perspective during the review process.

Decision Confidence: Self-reported confidence levels remained stable across conditions, with the Single-Output group averaging ($M = 3.13$) and the Dual-Output group averaging ($M = 3.00$). This indicates that while the dual interface invites comparison, it neither induces “analysis paralysis” nor provides a false sense of higher confidence regarding the final merge decision.

Difficulty (Cognitive Load): We evaluated the effects of Task Type (Secret Storage vs. User Creation) and Output Condition (Single vs. Dual) on perceived difficulty using a two-way Analysis of Variance (ANOVA). As anticipated for a pilot of this size, the overall model did not reach statistical significance ($F = 1.33, p = 0.31$). Consequently, we report directional trends for the individual factors:

- **Task Type:** The specific scenario did not significantly predict difficulty ratings ($F = 0.67, p = 0.43$). Detailed analysis suggests that this results from a distinct anchoring effect (Tversky and Kahneman, 1974): 87.5% of participants (7 out of 8) rated their first task as a ‘3’ regardless of the content. Variation emerged only in the second task, where participants correctly identified “Secret Storage” ($M = 3.25$) as more complex than “User Creation” ($M = 2.50$).
- **Number of LLMs:** Participants rated the Dual-Output condition ($M = 3.25, SE = 0.22$) as more difficult than the Single-Output condition ($M = 2.75, SE = 0.22$). While this difference suggests an increase in cognitive load, it did not reach statistical significance ($F = 2.67, p = 0.13$).
- **Interaction:** We found no significant interaction between task type and the number of LLMs ($F = 0.67, p = 0.43$).

Although limited by statistical power, these results confirm that our experimental design successfully captures variance in response difficulty. The observed trends imply that reviewing parallel outputs imposes a measurable cognitive tax. We will examine this hypothesis with a fully powered sample in the main study.

5.3. Qualitative Video Analysis: Codebook Validation

To assess the fidelity of our behavioural tracking, we conducted a preliminary screen-based interaction analysis on a subset of pilot recordings. The primary objective was methodological validation: determining if the proposed micro-interaction codebook accurately captures developer intent, or if specific codes require consolidation, expansion, or redefinition before the full-scale investigation.

Refining the Orientation Phase: Our initial protocol assumed the session began with `ORI_ISSUE` (reading the specific task). However, video analysis revealed a critical precursor behaviour. Participants frequently navigated to the repository’s main branch to review global instructions, particularly those on testing protocols and environmental setup. Furthermore, participants often returned to this documentation during the task when encountering verification blockers.

We propose adding a new code, `ORI_DOCS` (Documentation Review), to capture engagement with repository-level instructions distinct from issue-specific descriptions. This distinction is vital because frequent regression to `ORI_DOCS` may indicate that the task is unclear, whereas regression to `ORI_ISSUE` indicates the requirements are being checked.

Consolidating Exploration Metrics: The pilot revealed that the distinction between `EXP_FAST` (skimming) and `EXP_JUMP` (navigating) is largely semantic in small code-

bases. Because the generated files are relatively short, “fast scrolling” is functionally identical to “jumping” to a specific endpoint or function definition. We recommend merging these into a single `EXP_NAV` code (Structural Navigation).

Conversely, `EXP_SLOW` (reading) proved difficult to distinguish from `REV_HIGHLIGHT`. Participants often used the cursor to trace reading lines without clicking to highlight, or conversely, moved the mouse away to unblock text while reading intently. To capture these nuances, we propose a new `REV_DWELL` code that defines the visual dwell state where the cursor is either tracing text or idling in white space with periods of viewport stability (> 3 seconds).

Test Suite Review: A significant observed behaviour was the scrutiny of pre-existing test suites. Participants frequently reviewed test files not just to run them (`REV_TEST`), but to understand the intended functionality and potential security coverage. Unlike `ORI_ISSUE`, in which the participant reads a high-level task description in prose, these interactions involved reading code to understand the precise technical contract. We have decided to treat this as a distinct cognitive activity and recommend introducing the `REV_SPEC` code. This captures instances where the user reads test files to understand the intended behaviour, distinguishing it from reviewing the implementation.

Strategic Refinement: Finally, our video analysis exposed a critical limitation in the pilot stimuli: both LLM-generated options were uniformly insecure. Consequently, the comparative workflow failed to elicit the intended security awareness; participants limited their comparison to functional logic because neither option offered a security advantage to contrast with the other. To correct this in the full-scale investigation, we propose a “Differential Security” strategy. Rather than presenting two equally vulnerable options, the system will generate outputs with distinct, non-overlapping security patches (e.g., Output A remediates SQL Injection, while Output B remediates Access Control). This approach heightens ecological validity. It reflects the inconsistency of real-world LLM security while creating a functional incentive for comparison. The reviewer must identify the partial fix in one option and actively apply it to the other to engineer a fully secure solution.

6. Discussion

Our pilot study served two distinct purposes: to obtain a preliminary signal on the efficacy of the “Parallel Pull Request” workflow and, more critically, to stress-test the experimental protocol. The results provide a nuanced picture of the trade-offs between cognitive load and security scrutiny.

6.1. Interpretation of Pilot Findings

Quantitatively, the dual-output interface did not produce a statistically significant improvement in security outcomes within this sample ($N = 8$). However, the secondary metrics offer a compelling explanation for this null result. Participants rated the dual-output condition as more difficult ($M = 3.25$) and took longer to complete it ($M = 48.77$ min), confirming that the interface successfully triggered “cognitive forcing” and it disrupted the seamless flow of review. Yet, this increased friction did not translate into better vulnerability detection.

We attribute this disconnect to the nature of the dual output. In the pilot, both AI-generated options contained identical classes of vulnerabilities, i.e. Option A was as insecure as Option B. Consequently, while the interface forced participants to compare the code, the comparison yielded no new security information. This rendered the cognitive effort of comparison unproductive friction rather than productive scrutiny. Furthermore, the significant order effect ($p = 0.027$) indicates that fatigue and learning curves played a larger role than the interface itself, with participants rushing the second task regardless of the condition.

6.2. Methodological Validation

The pilot successfully validated several core components of our methodology while highlighting critical areas for refinement.

- **Ecological Validity:** The simulated GitHub environment proved robust. Participants navigated the interface intuitively, using standard workflow features without confusion, thereby validating the decision to use a simulated environment over a static survey.
- **Metric Sensitivity:** We had to bifurcate *Security Performance* into *Discovery* and *Resolution* due to the participants’ limited familiarity with the FastAPI framework. A purely binary metric would have masked the fact that participants often spotted issues (*Discovery*) even if they lacked the syntax knowledge to fix them (*Resolution*).
- **Experimental Control:** The Latin Square design successfully isolated the cognitive load of the interface from the difficulty of the tasks, as evidenced by the consistent difficulty ratings across scenarios.

6.3. Strategic Refinements for Full-Scale Investigation

Based on these insights, we define three pivotal refinements for the full-scale study to ensure rigorous evaluation of the research questions.

1. Implementation of Differential Security: To convert unproductive friction into valid scrutiny, we will implement

the *Differential Security* strategy. Future PRs will feature non-overlapping security patches (e.g., Option A fixes SQL Injection; Option B fixes Access Control). This ensures that the act of comparison provides tangible informational value, incentivizing the reviewer to synthesize a secure solution from the two imperfect options.

2. Target Population and Generalizability: The pilot participants (students) struggled with framework-specific syntax, confounding the measurement of security awareness. The full study will recruit industry professionals with demonstrated backend experience. This addresses the generalizability concern raised regarding diverse organizational review practices; while workflows differ, the fundamental cognitive mechanism of code comparison remains a universal human factor.

3. Protocol Tightening: To mitigate the fatigue effects observed in the pilot, we will enforce a strict 30-minute time limit per task. Analysis will shift from raw *Time to Completion* to *Survival Analysis* (success rate per unit of time). Additionally, we will integrate the NASA-TLX assessment to mathematically quantify the **Difficulty** ratings, allowing us to distinguish between useful mental effort and frustration.

7. Conclusion

As Large Language Models (LLMs) increasingly automate code generation, the human developer’s role shifts from author to auditor. This study proposed a “Parallel Pull Request” interface designed to support this transition by cognitively forcing comparative evaluation. While our pilot study showed that the interface successfully increased cognitive engagement, it also revealed that engagement alone is insufficient without distinct security cues in the options. By refining our protocol to incorporate *Differential Security* and recruiting domain experts, the upcoming full-scale investigation aims to definitively determine whether this interaction model can close the reliability gap, thereby transforming the “human-in-the-loop” from a passive bottleneck into an active security layer.

8. Author Contribution

Preparation Stage Felix, drawing on his technical background, led the preparation of all technical components of the project, including coding tasks, test suite, etc. Ashu took the responsibility for the experiment setup, including drafting the ethics protocol and conducting candidate screening.

Experiment Stage Both members jointly conducted the study sessions and coordinated participant interaction.

Data Analysis & Report Drafting Felix tested the participants’ submission and collected data through systematically

evaluating the produced code outputs, PR review comments, and audio transcripts. Ashu carefully reviewed selected user-study screen recordings and identified future amendment directions before the full-scale study for the micro-level interaction code discussed in the paper. Felix conducted the initial round of data collection and analysis, and Ashu, drawing on his HCI background and prior user study experience, extended the analysis by introducing additional statistical methods and providing justification for their application. Both authors jointly contributed to the report sections corresponding to the components they contributed to, with shared responsibility for integration and refinement.

Data Availability

A replication package, available at (<https://doi.org/10.5281/zenodo.17851012>), including our Ethics Clearance Certificate, detailed tables showing participant demographics, survey responses, and quality of produced code. User study recording transcripts and participant submissions are omitted because they cannot be reliably de-identified.

References

- Asare, O., Nagappan, M., and Asokan, N. (2024). A user-centered security evaluation of copilot. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–11.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Barke, S., James, M. B., and Polikarpova, N. (2022). Grounded copilot: How programmers interact with code-generating models.(2022). *CoRR arXiv*, 2206.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. (2021). Evaluating large language models trained on code.
- Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., and Lahiri, S. K. (2024). Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2023). Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*.
- Letovsky, S. (1987). Cognitive processes in program comprehension. *Journal of Systems and software*, 7(4):325–339.
- Littman, D. C., Pinto, J., Letovsky, S., and Soloway, E. (1987). Mental models and software maintenance. *Journal of Systems and Software*, 7(4):341–355.
- OWASP Foundation (2025a). A01: Broken access control – OWASP top 10:2021.
- OWASP Foundation (2025b). A06: Insecure design – OWASP top 10:2025.
- Perry, N., Srivastava, M., Kumar, D., and Boneh, D. (2023). Do users write more insecure code with ai assistants? In *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, pages 2785–2799.
- Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., and Dolan-Gavitt, B. (2023). Lost at c: A user study on the security implications of large language model code assistants. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2205–2222.
- The MITRE Corporation (2024). 2024 CWE top 25 most dangerous software weaknesses.
- Tversky, A. and Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases: Biases in judgments reveal some heuristics of thinking under uncertainty. *science*, 185(4157):1124–1131.
- Vaithilingam, P., Zhang, T., and Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*, pages 1–7.
- Vero, M., Mündler, N., Chibotaru, V., Raychev, V., Baader, M., Jovanović, N., He, J., and Vechev, M. (2025). Baxbench: Can llms generate correct and secure backends? *arXiv preprint arXiv:2502.11844*.
- Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., and Aftandilian, E. (2022). Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pages 21–29.